

WatchTower

Multi-Server Remote Health Monitor

Mini projet - Theorie des systemes d exploitation & SE Windows/Unix/Linux

Script Bash principal: watchtower

Livrables: PDF, PPTX, script Bash, Dockerfile, config, helper C pthread

1. Besoin identifie

Un administrateur systeme doit souvent se connecter manuellement a plusieurs serveurs pour verifier CPU, RAM, disque et services. Cette methode prend du temps, cree des erreurs de saisie et retarde la detection des incidents. WatchTower automatise ce controle depuis un seul terminal.

2. Solution proposee

WatchTower lit un fichier servers.conf, detecte le type de cible, collecte les metriques et affiche un tableau live. Les conteneurs Docker locaux utilisent docker stats depuis la machine hote. Les serveurs Linux reels utilisent SSH et les commandes Unix. Le serveur Windows utilise SSH et PowerShell.

3. Syntaxe d execution

```
./watchtower [options] -c servers.conf
```

4. Options implementees

Option	Role	Directive couverte
-h	Affiche la documentation	Help Linux-style
-c	Fichier de configuration obligatoire	Parametre obligatoire
-s	Execution dans des sous-shells Bash	Subshell
-f	Processus fils en arriere-plan avec wait	Fork / processus fils
-t	Helper C pthread_create/pthread_join + workers Bash	Threads POSIX via C
-l	Repertoire de journalisation	history.log configurable
-r	Reset logs/cache, admin seulement	Privilege administrateur
-b	Archive tar.gz de history.log	Archivage/compression
-i	Intervalle de rafraichissement	Parametre optionnel
-a	Seuils cpu/ram/disk	Validation + alertes

5. Architecture

```
watchtower (Bash)
|-- parse servers.conf
|-- Docker cible locale -> docker stats --no-stream
|-- Linux serveur reel -> ssh + awk/free/df/uptime/systemctl
|-- Windows ARM VM -> ssh + PowerShell Get-CimInstance
|-- dashboard terminal -> tableau live
|-- history.log -> INFOS / ERROR horodates
|-- -t helper C -> pthread_create / pthread_join
```

6. Journalisation

Le script utilise history.log avec le format impose. Si /var/log/watchtower est inaccessible sans sudo, WatchTower bascule vers ./logs et ce comportement est documente.

```
yyyy-mm-dd-hh-mm-ss : username : INFOS : message
yyyy-mm-dd-hh-mm-ss : username : ERROR : message
```

7. Gestion d erreurs

Code	Signification
100	Option inconnue
101	Config obligatoire manquante
102	Fichier config introuvable

103	Format config invalide
104	Echec SSH
105	Echec Docker
106	Permission/admin
107	Dependance manquante

8. Tableau de bord

Server	OS	CPU	RAM	DISK	PING	FAIL	SVC	Status
server1	linux	12%	5%	1%	local	0	1/1	HEALTHY
server2	linux	92%	6%	1%	local	0	1/1	CRITICAL
server5	windows	8%	59%	27%	N/A	0	1/1	HEALTHY

9. Scenarios de test

Scenario	Commande	Objectif
Leger	<code>./watchtower -s -c servers-light.conf</code>	2 serveurs Docker, mode subshell
Moyen	<code>./watchtower -t -c servers-medium.conf</code>	3 serveurs Docker, helper pthread + workers
Lourd	<code>./watchtower -f -c servers.conf</code>	4 Docker + Windows, stress CPU sur server2

Stress CPU:

```
docker exec server2 bash -c "yes > /dev/null &"
docker stats --no-stream server1 server2 server3 server4
docker exec server2 kill yes
```

10. Repartition proposee

Personne 1: options, config, erreurs, logging. Personne 2: Docker/Linux metrics et demo stress. Personne 3: Windows, dashboard, rapport/PPT et presentation.